대한상공회의소
서울기술교육센터

SPI, I2C 설계
UVM 검증

+ Demo Video

발표자 김지홍

Chapter 0

목차

: Contents

Instruction

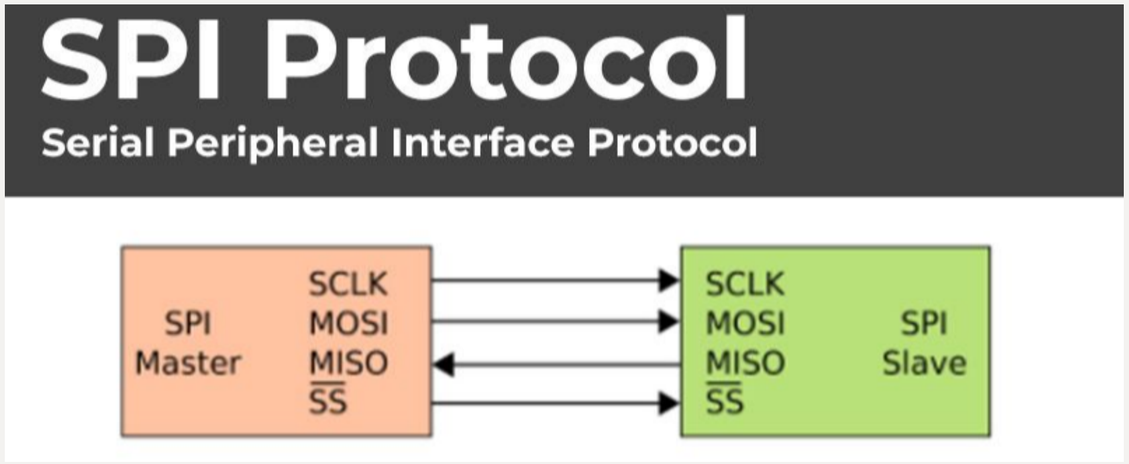
Chapter 1

개요

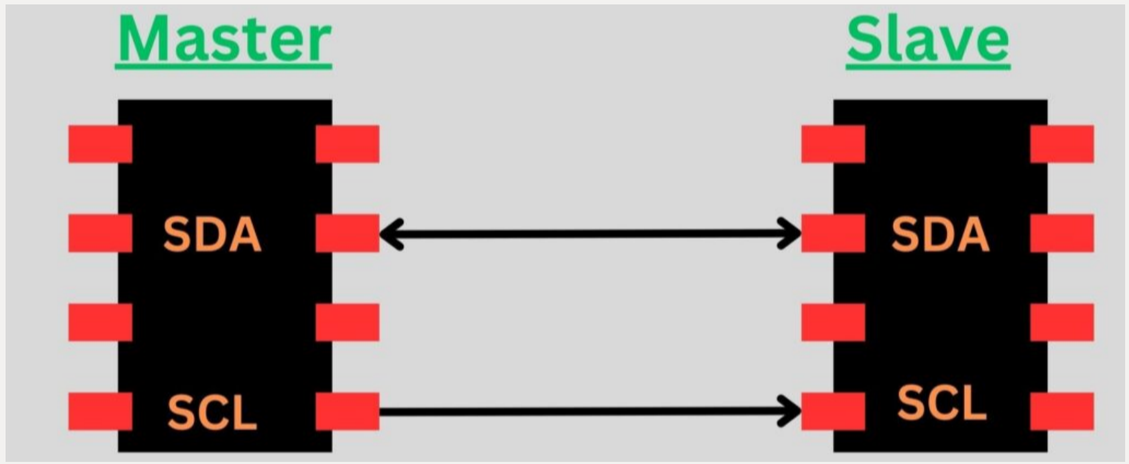
: Instruction

“

대표적인 Serial Interface : UART, SPI, I2C



(*1*) SPI protocol

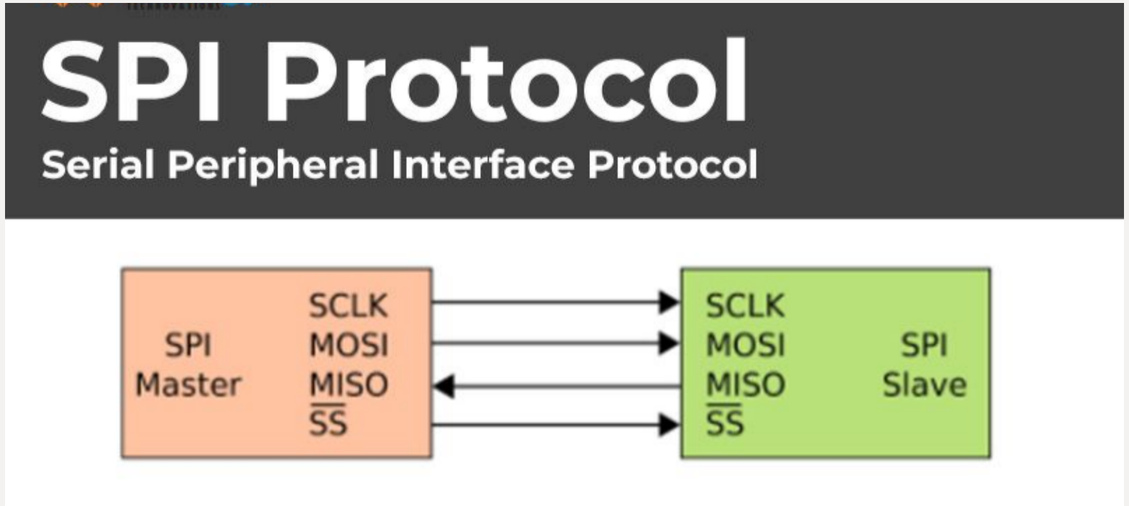


(*2*) I2C protocol

Chapter 1

개요

: Instruction

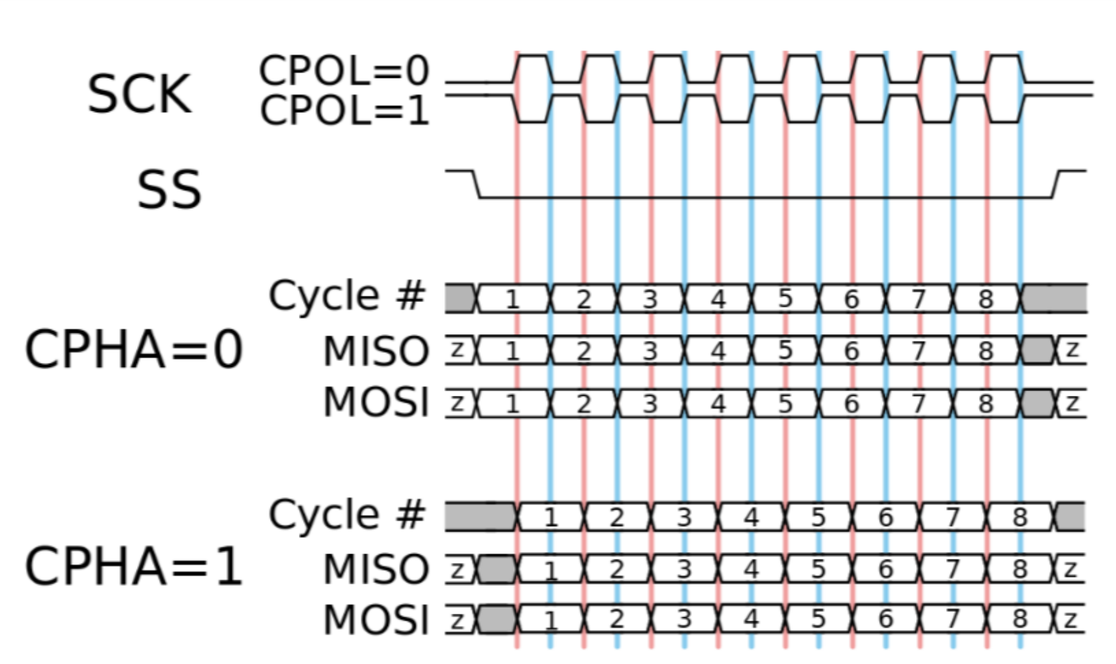


- *Protocol* — *SPI - Serial Peripheral Interface*
- *Character* — Synchronous, Full Duplex, SS/CS
  Master-slave, 1 : N
- *PORT* — SCLK - 동기화를 위한 Master의 출력 클럭
  MOSI - Master Out Slave In
  MISO - Master In Slave Out
  SS/CS - Slave Select / Chip Select

Chapter 1

개요

: Instruction



- *Protocol* — *SPI - Serial Peripheral Interface*

- *Mode* — CPOL, CPHA에 따라 4가지 Mode 존재, 데이터를 쓰는 시점과 읽는 시점을 Master와 Slave가 똑같이 맞추는 과정

첫번째 비트 나머지 비트

Mode 0 (0,0): CPOL(SCLK) = 0 → Falling(SS) - write, Rising(SCLK) – read, Falling(SCLK) - write

Mode 1 (0,1): CPOL(SCLK) = 0 → Rising(SCLK) - write, Falling(SCLK) – read, Rising(SCLK) - write

첫번째 비트 나머지 비트

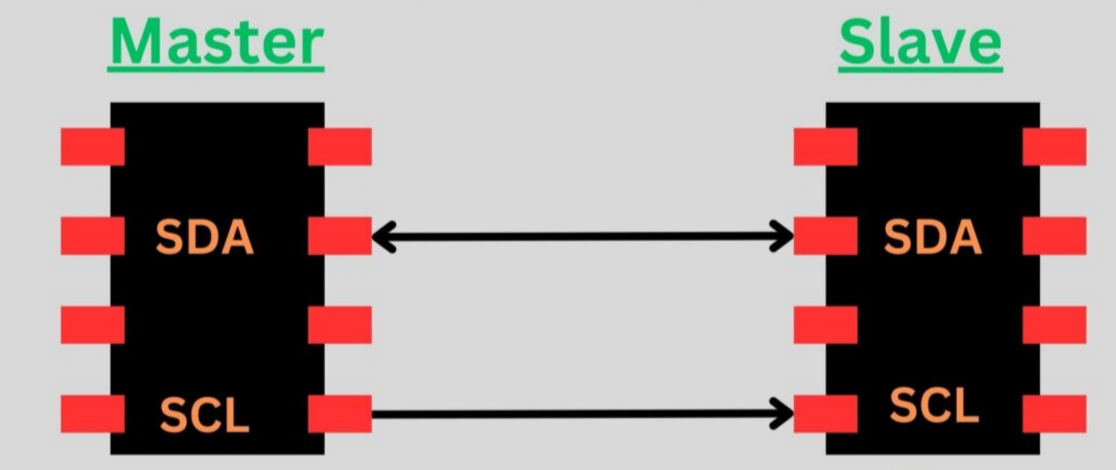
Mode 2 (1,0): CPOL(SCLK) = 1 → Falling(SS) - write, Falling(SCLK) – read, Rising(SCLK) – write

Mode 3 (1,1): CPOL(SCLK) = 1 → Falling(SCLK) - write, Rising(SCLK) – read, Falling(SCLK) - write

Chapter 1

개요

: Instruction

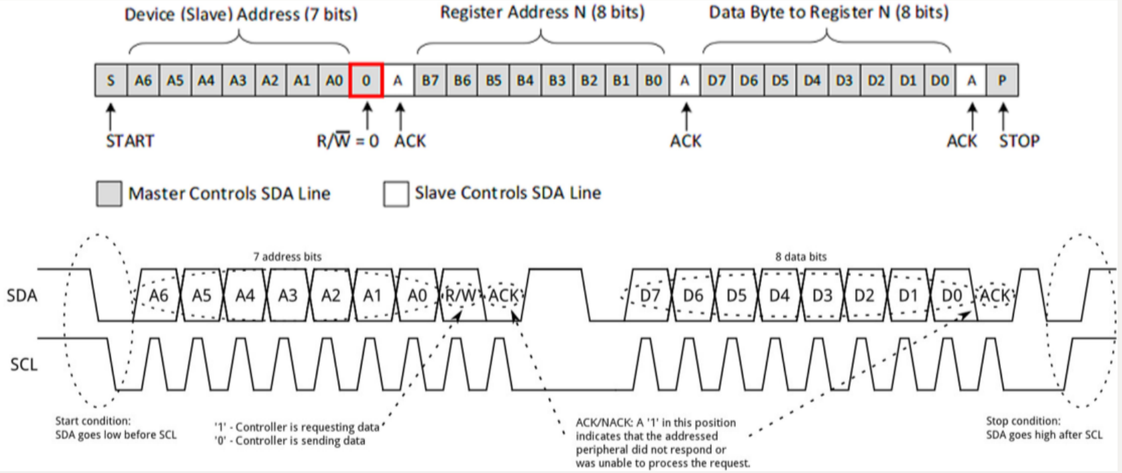


- *Protocol* — *I2C - Inter-Integrated Circuit*
- *Character* — Synchronous, Half Duplex, Address, Master-slave, 1 : N
- *PORT* — SDA - Serial Data
  SCL - 동기화를 위한 Master의 출력 클럭

Chapter 1

개요

: Instruction



START : SCLK = 1 → SDA - Falling

STOP : SCLK = 1 → SDA - Rising

Data 0 : SCLK = 1 → SDA = 0(LOW) 유지

Data 1 : SCLK = 1 → SDA = 1(HIGH) 유지

- *Protocol* — *I2C - Inter-Integrated Circuit*

- *Process*

Write
: Start signal → {Slave address[7bit], write[1'b0]} → Ack(from slave) → Tx data[8bit] Write → Ack(from slave) → Stop

Read
: Start signal → {Slave address[7bit], read[1'b1]} → Ack(from slave) → Rx data[8bit] Read → Nack(from master) → Stop

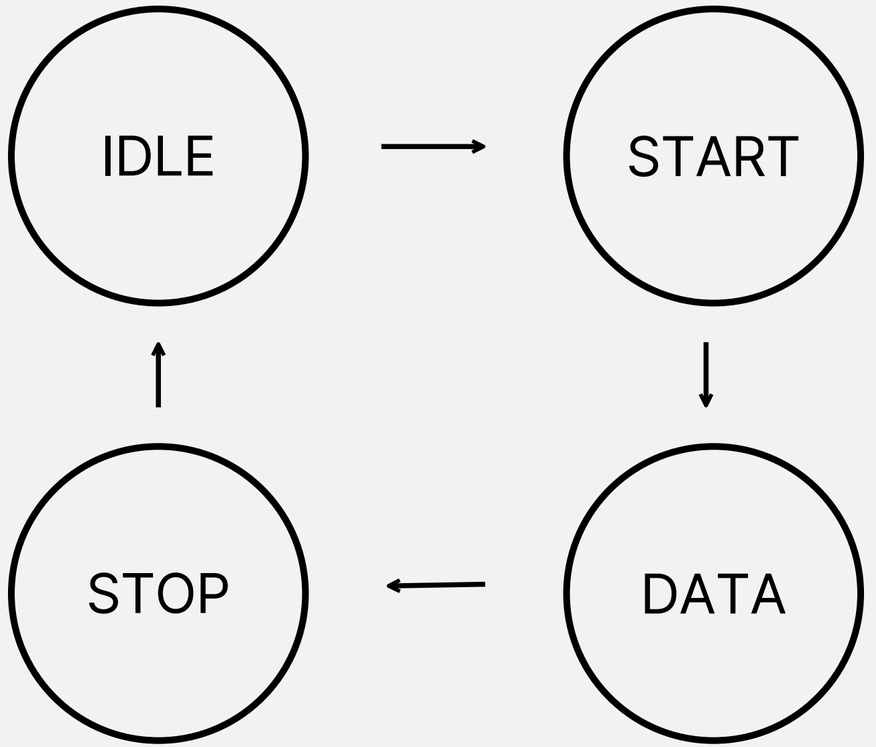
SPI slave 설계 및 Demo Video

Chapter 2

SPI 설계

: SPI slave 설계 및 Demo Video

SPI Master

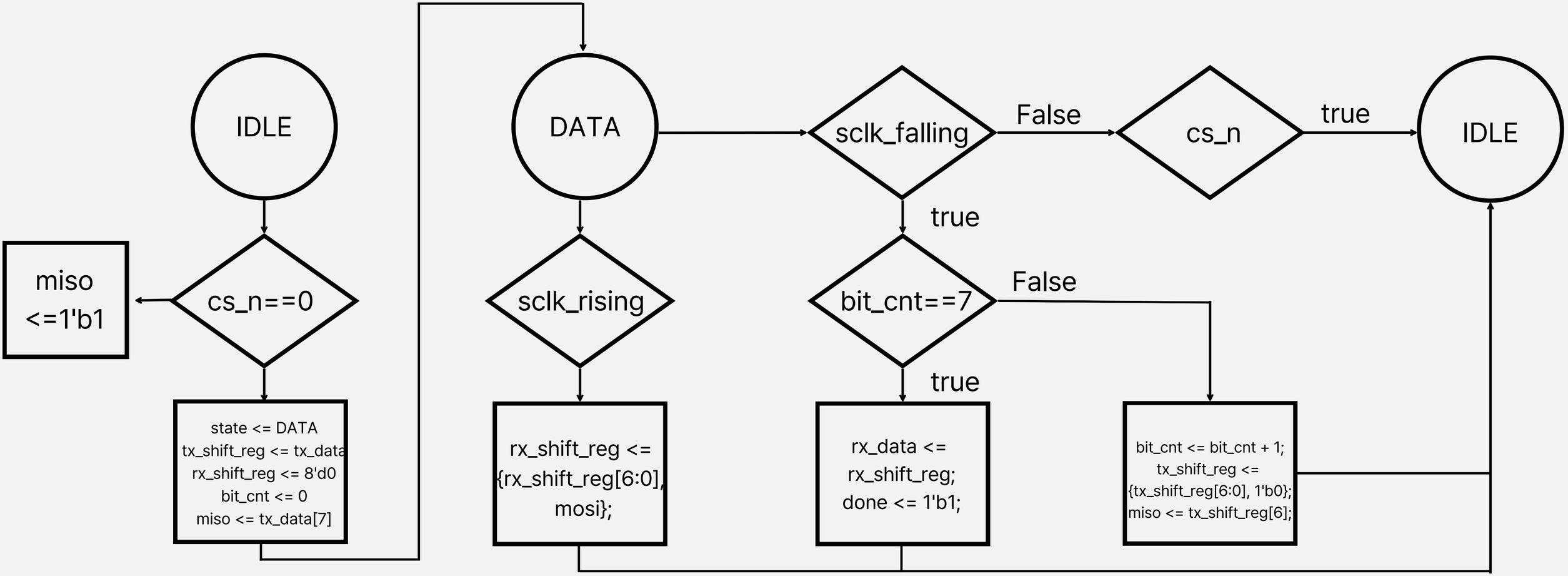


Chapter 2

SPI 설계

: SPI slave 설계 및 Demo Video

SPI Slave

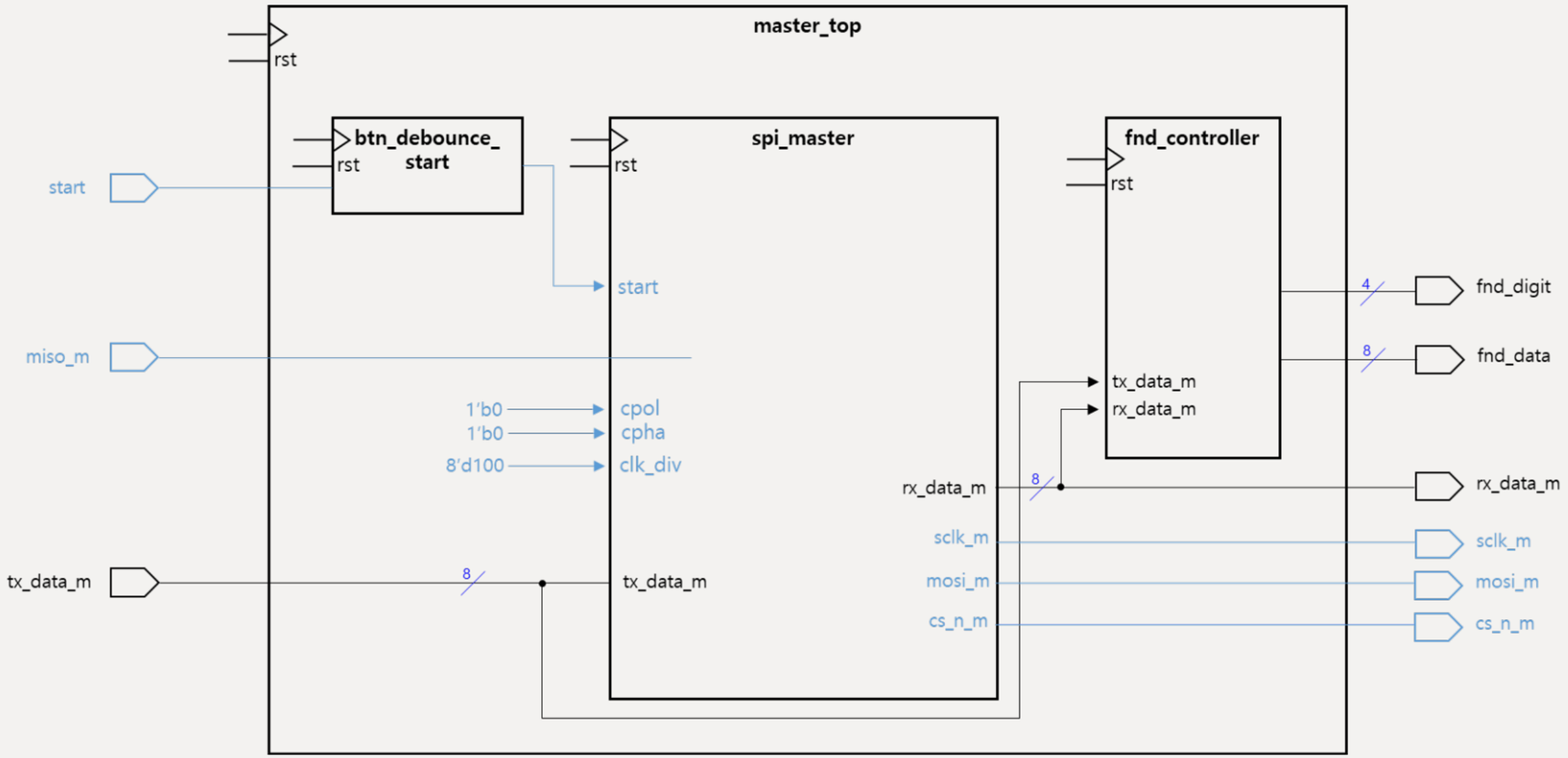


Chapter 2

SPI 설계

: SPI slave 설계 및 Demo Video

SPI Master

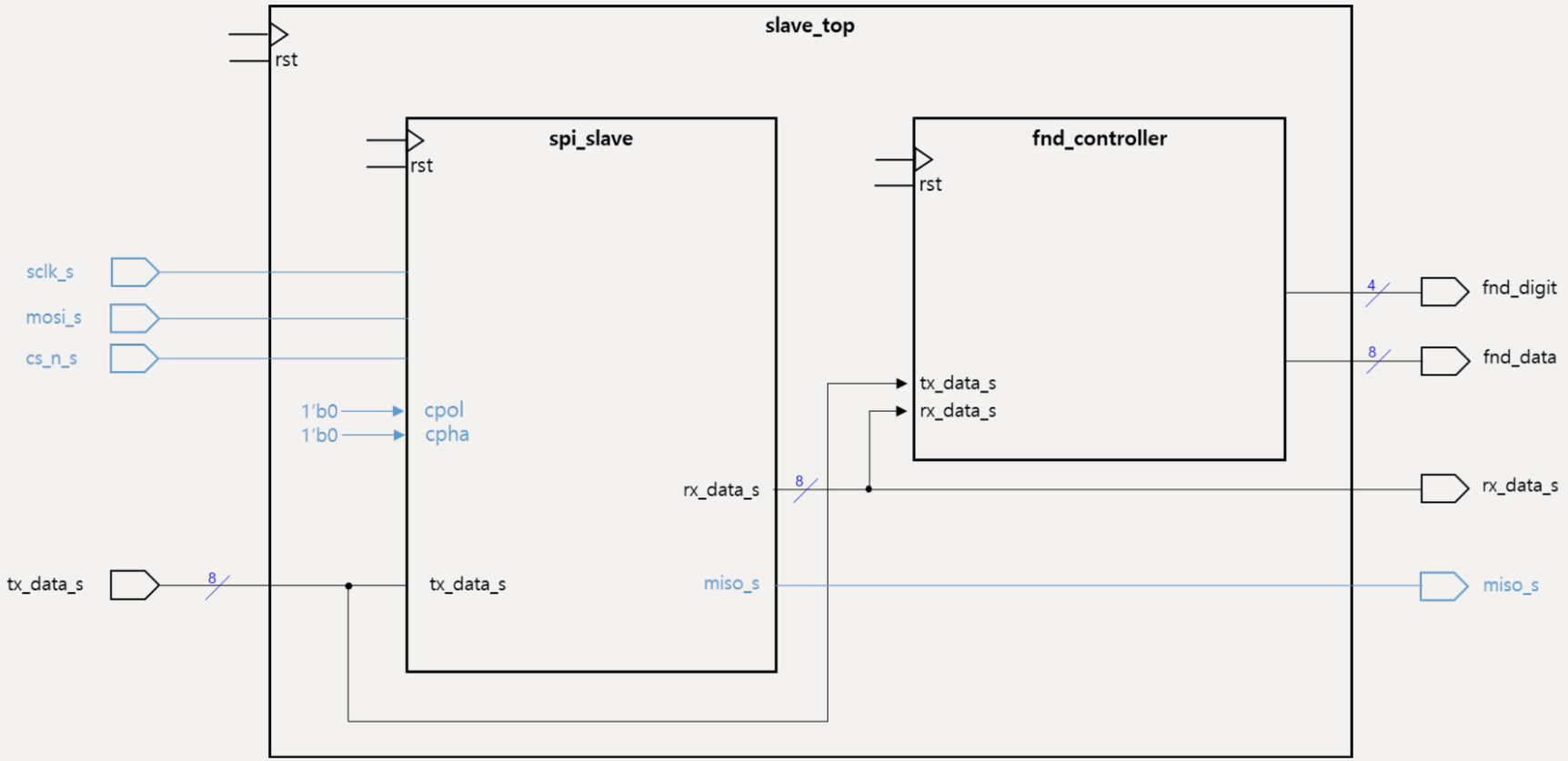


Chapter 2

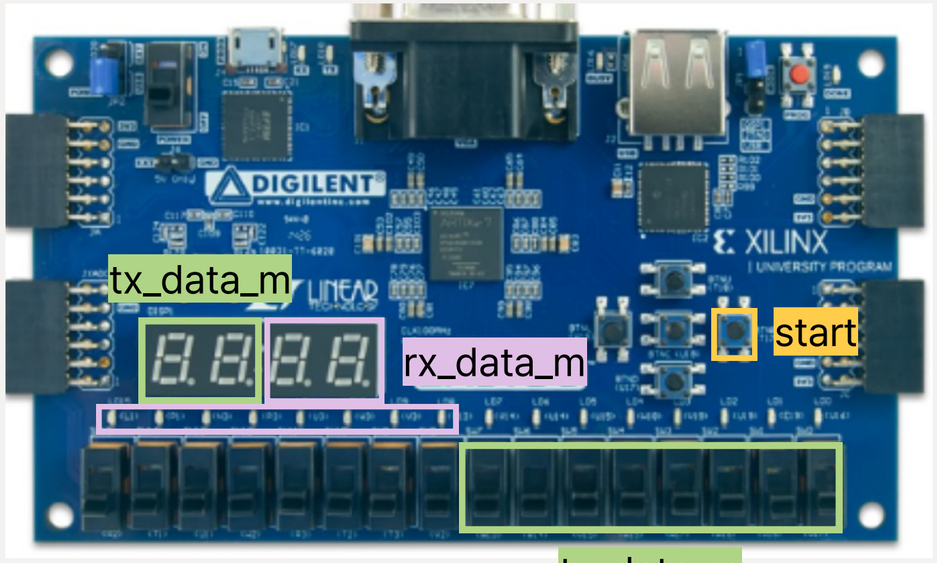
SPI 설계

: SPI slave 설계 및 Demo Video

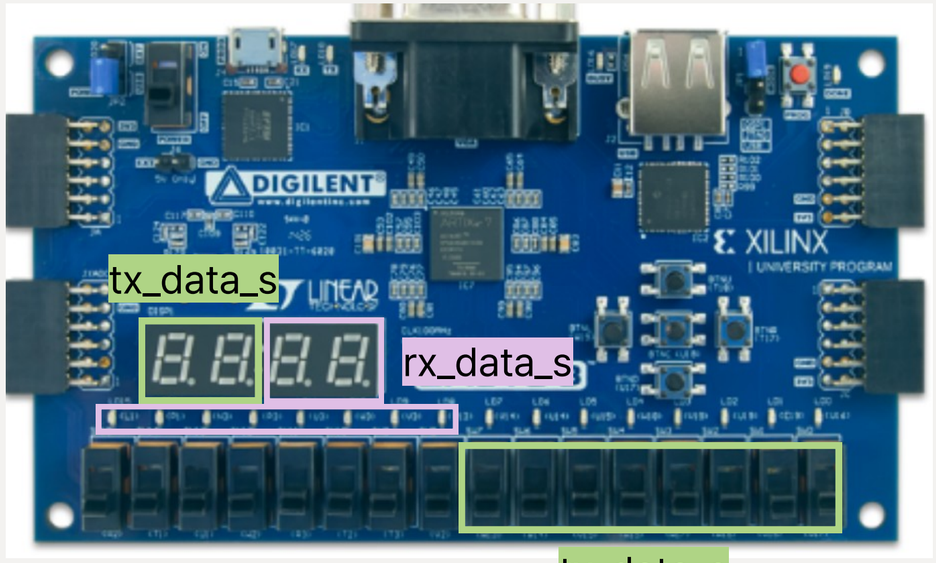
SPI Slave



: SPI slave 설계 및 Demo Video



< Master >



< Slave >

Chapter 2

SPI Demo

: SPI slave 설계 및 Demo Video

SPI Demo Scenario

Write : Slave에 쓰고 싶은 8비트 data를 Master에서 switch를 조작해 tx_data로 입력
→ start 버튼 → write (Slave fpga의 FND와 LED에 출력)

Read : Master에 읽고 싶은 8비트 data를 Slave에서 switch를 조작해 tx_data로 입력
→ start 버튼 → read (Master fpga의 FND와 LED에 출력)

Chapter 2

SPI Demo

: SPI slave 설계 및 Demo Video

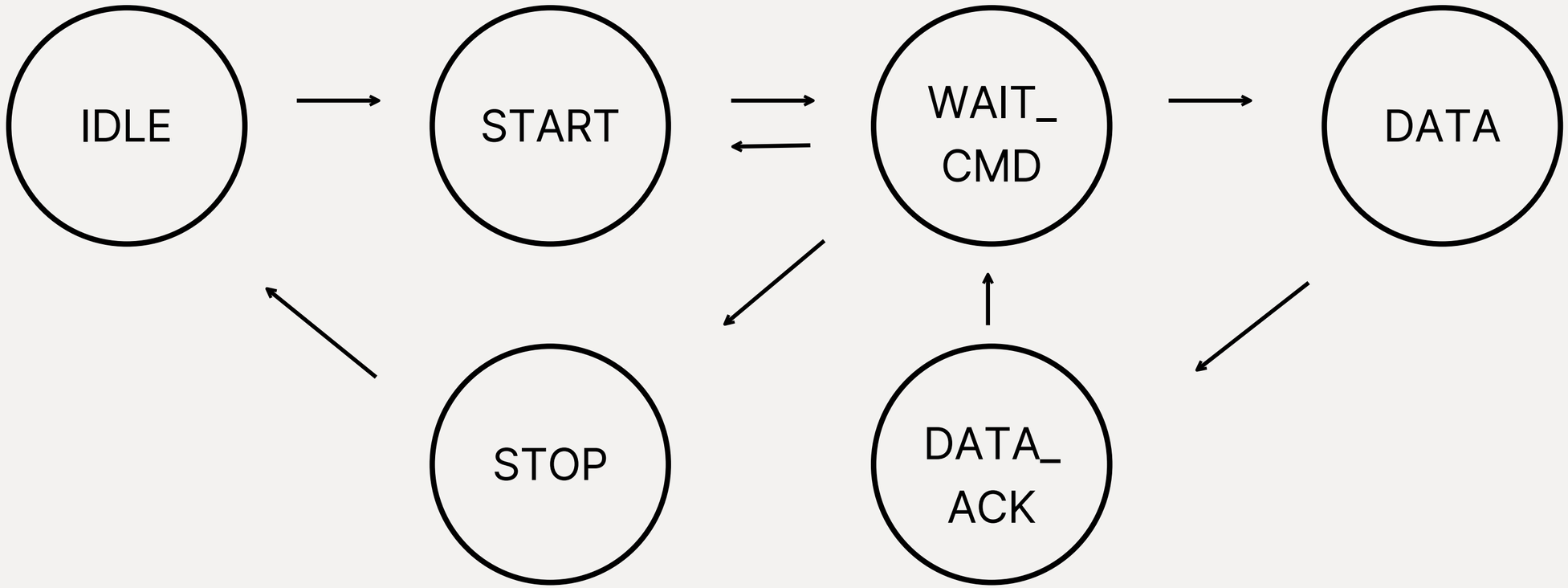
I2C slave 설계 및 Demo Video

Chapter 3

I2C 설계

: I2C slave 설계 및 Demo Video

I2C Master

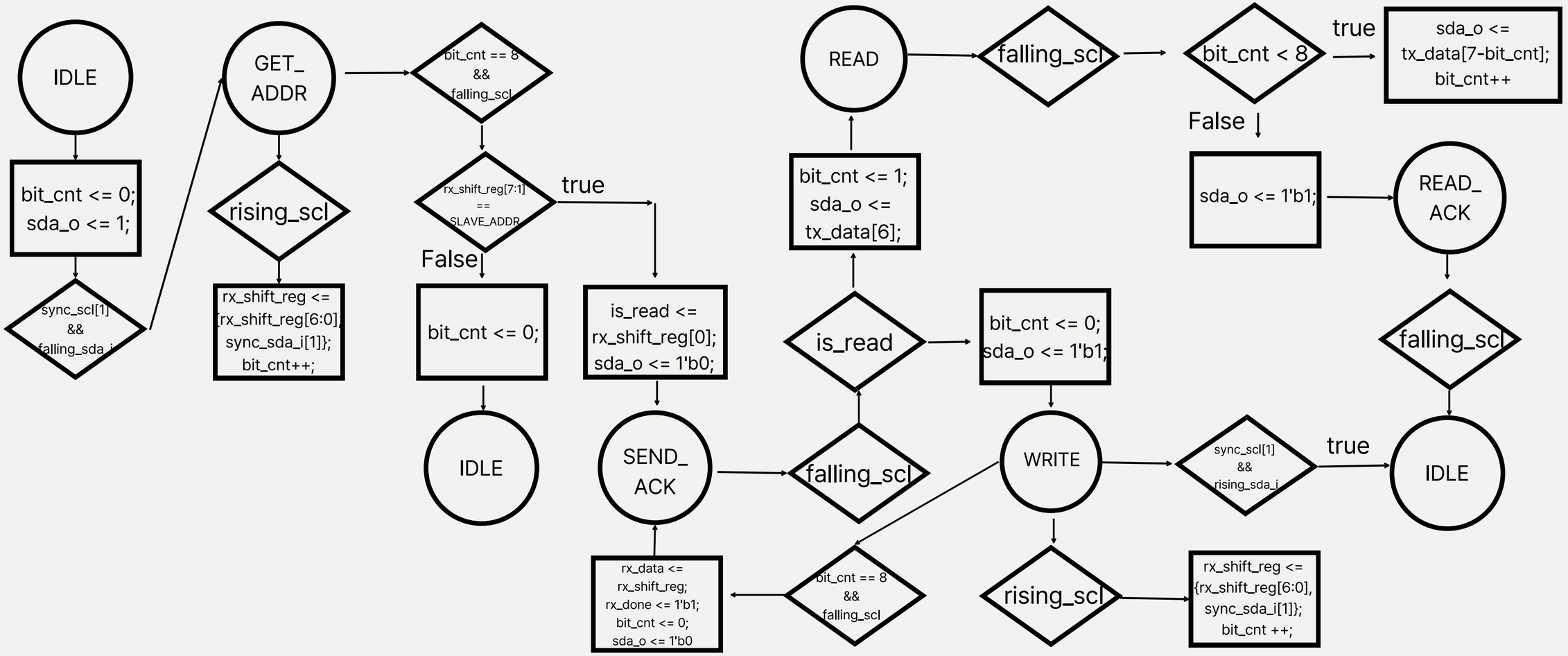


Chapter 3

I2C 설계

: I2C slave 설계 및 Demo Video

I2C Slave

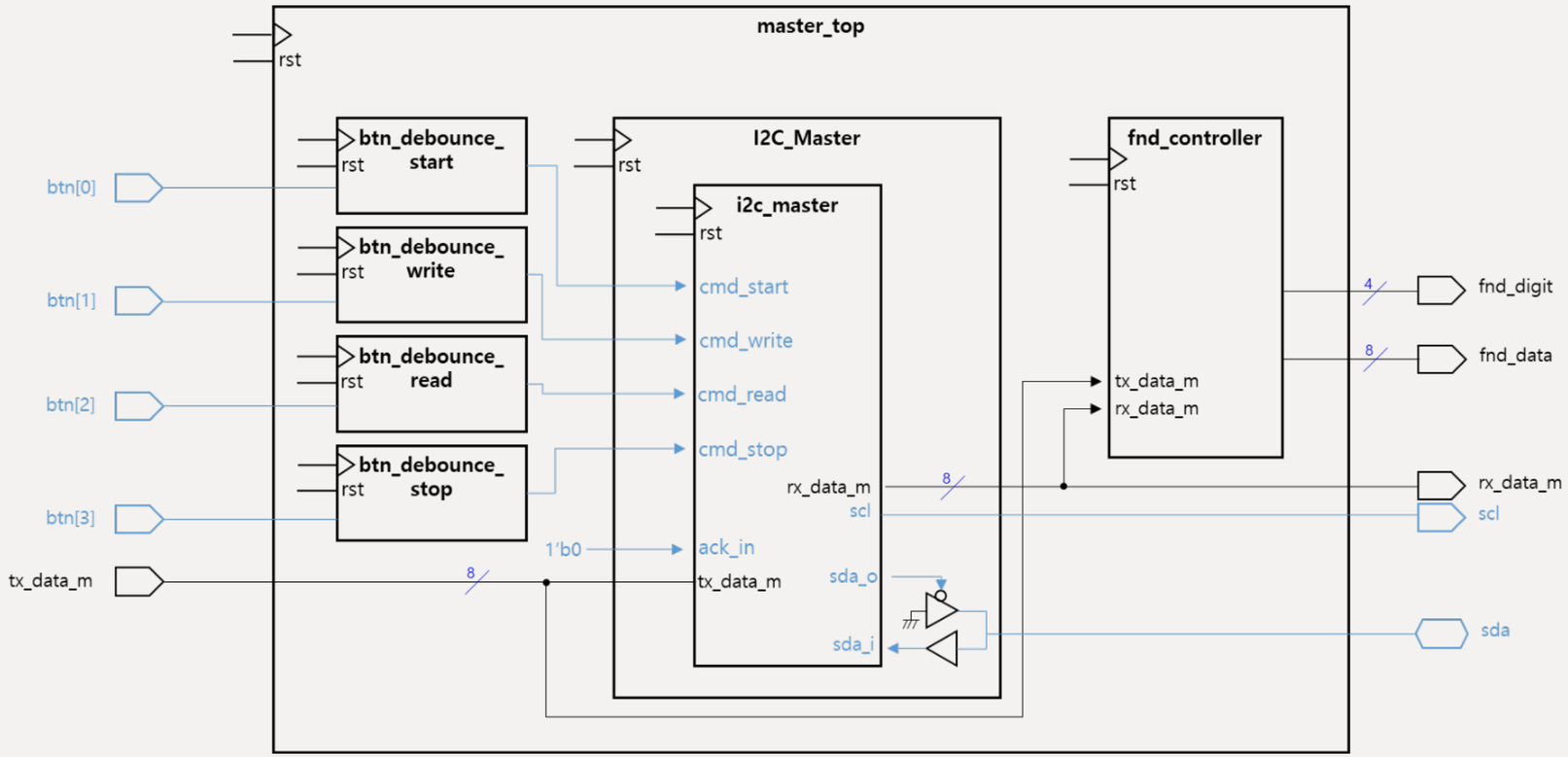


Chapter 3

I2C 설계

: I2C slave 설계 및 Demo Video

I2C Master

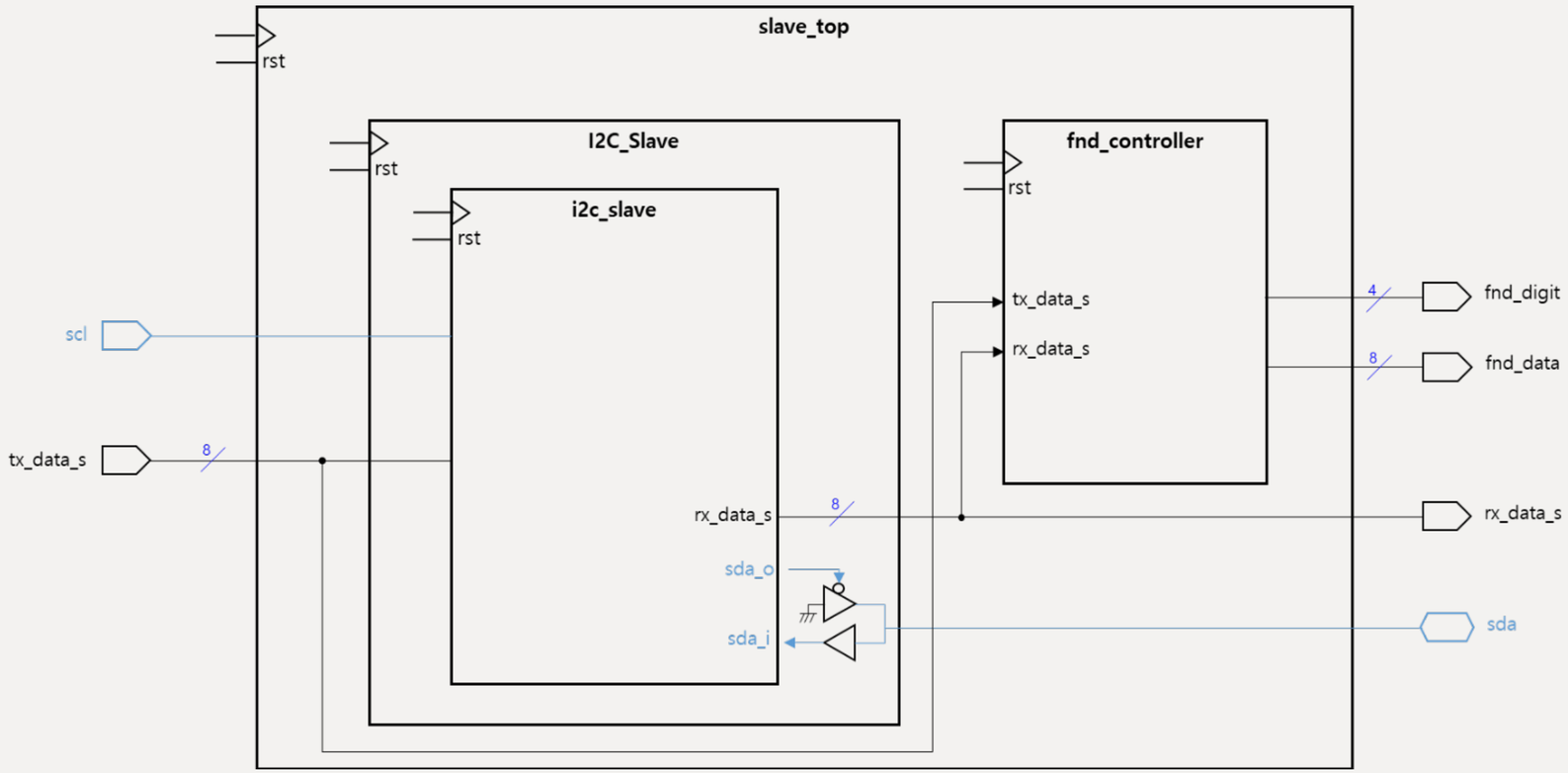


Chapter 3

I2C 설계

: I2C slave 설계 및 Demo Video

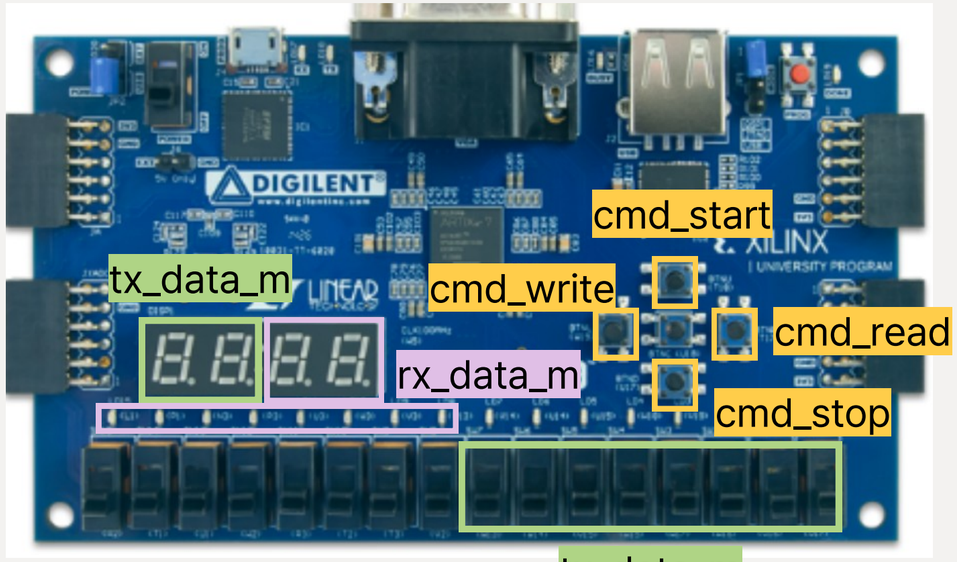
I2C Slave



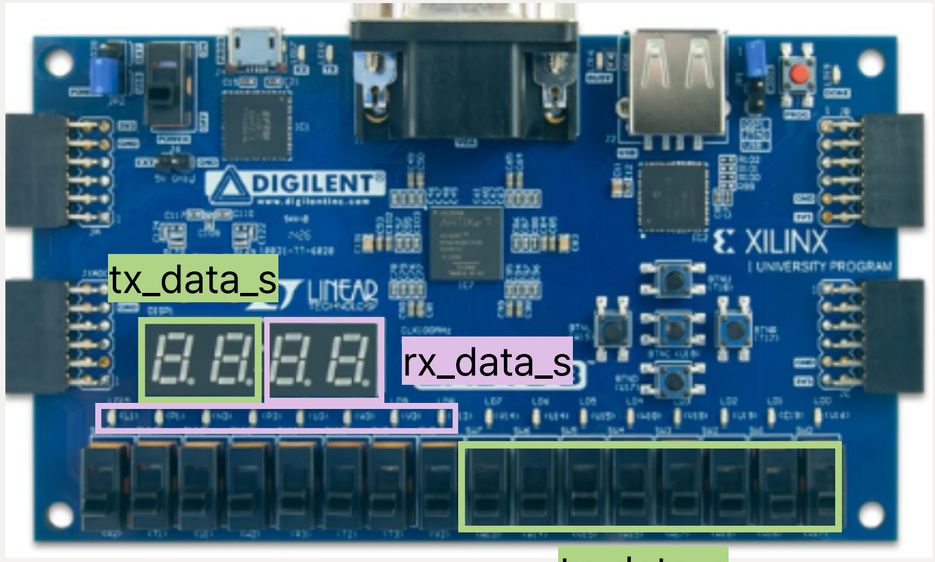
Chapter 3

I2C Demo

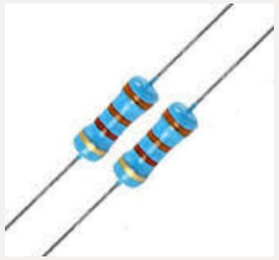
: I2C slave 설계 및 Demo Video



< Master >



< Slave >



< Resistor >

: I2C slave 설계 및 Demo Video

I2C Demo Scenario

Write : start → “{slave address, 0(write)}" 를 switch를 조작해 tx_data로 입력
→ write → Slave에 쓰고 싶은 8비트 data를 Master에서 tx_data로 입력
→ write → stop

Read : start → “{slave address, 1(read)}" 를 switch를 조작해 tx_data로 입력
→ write → Master에 쓰고 싶은 8비트 data를 Slave에서 tx_data로 입력
→ read → stop

Chapter 3

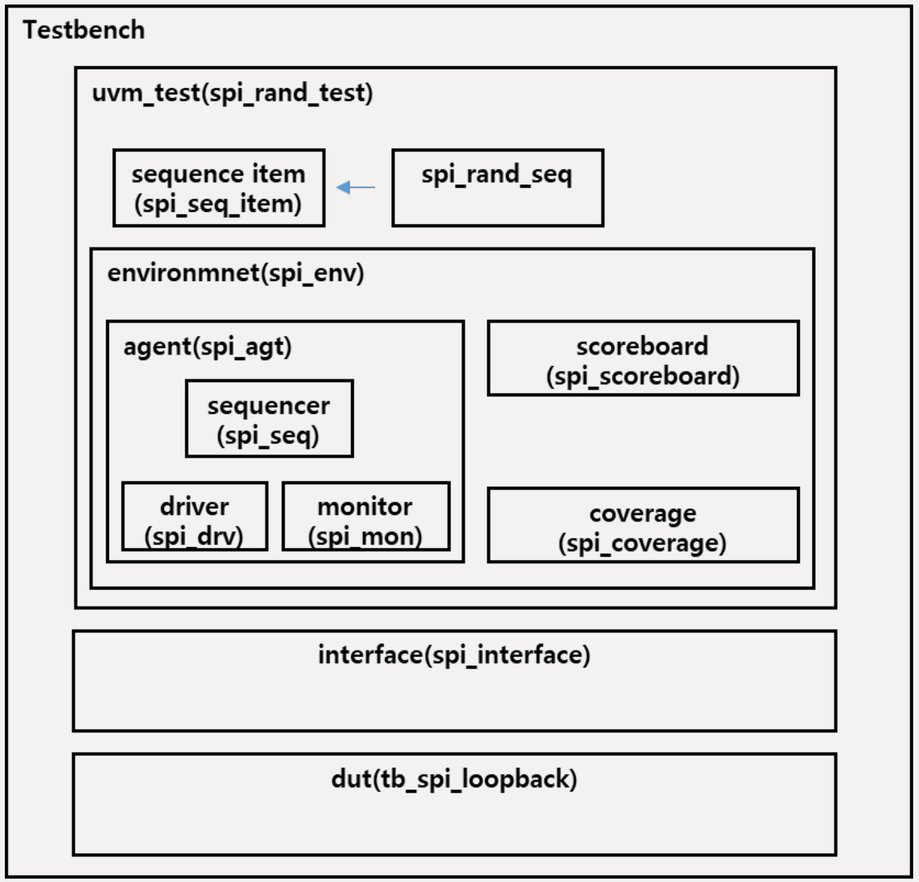
I2C Demo

: I2C slave 설계 및 Demo Video

UVM

: UVM

SPI LoopBack




```
==========================================
        SPI SCOREBOARD SUMMARY
==========================================
 [Master->Slave]  Pass: 999, Fail: 0
 [Slave->Master]  Pass: 1000, Fail: 0
------------------------------------------
 Total Successful Transfers: 1999
==========================================
Test Result: PASSED
```

: UVM

SPI LoopBack - timing

```
task run_phase(uvm_phase phase);
    spi_seq_item item;
    vif.start <= 0;

    wait (vif.rst == 0);
    repeat (10) @(vif.drv_cb);

    forever begin
        seq_item_port.get_next_item(item);

        vif.cpol      <= item.cpol;
        vif.cpha      <= item.cpha;
        vif.tx_data_m <= item.tx_data_m;
        vif.tx_data_s <= item.tx_data_s;

        repeat (5) @(vif.drv_cb);

        vif.start <= 1'b1;
        @(vif.drv_cb);
        vif.start <= 1'b0;

        wait (vif.drv_cb.done_m === 1'b1);

        repeat (10) @(vif.drv_cb);

        item.rx_data_m = vif.drv_cb.rx_data_m;
        item.rx_data_s = vif.drv_cb.rx_data_s;

        seq_item_port.item_done();
    end
endtask
```

driver

```
task run_phase(uvm_phase phase);
    spi_seq_item item;
    forever begin
        @(vif.mon_cb);
        if (vif.mon_cb.done_m) begin
            //@(vif.mon_cb);
            item           = spi_seq_item::type_id::create("item");
            // 데이터 복사 로직 추가 (필수)
            item.cpol      = vif.mon_cb.cpol;
            item.cpha      = vif.mon_cb.cpha;
            item.tx_data_m = vif.mon_cb.tx_data_m;
            `uvm_info(get_type_name(), $sformatf("[Master_TX] tx_data_m = 0x%02h",
                                                 item.tx_data_m), UVM_HIGH)
            item.tx_data_s = vif.mon_cb.tx_data_s;
            `uvm_info(get_type_name(), $sformatf("[Slave_TX] tx_data_s = 0x%02h",
                                                 item.tx_data_s), UVM_HIGH)
            item.rx_data_m = vif.mon_cb.rx_data_m;
            `uvm_info(get_type_name(), $sformatf("[Master_RX] rx_data_m = 0x%02h",
                                                 item.rx_data_m), UVM_HIGH)
            item.rx_data_s = vif.mon_cb.rx_data_s;
            `uvm_info(get_type_name(), $sformatf("[Slave_RX] rx_data_s = 0x%02h",
                                                 item.rx_data_s), UVM_HIGH)
            ap.write(item);
        end
    end
endtask
```

monitor

: UVM

SPI LoopBack - Coverage

```
constraint mode0 {cpol == 1'b0; cpha == 1'b0;}
```

```
covergroup cg_spi;
    // CPOL, CPHA
    cp_cpol: coverpoint t_item.cpol {
        bins mode0 = {1'b0};
        illegal_bins others = {1'b1};
    }
    cp_cpha: coverpoint t_item.cpha {
        bins mode0 = {1'b0};
        illegal_bins others = {1'b1};}

    // Data (Master 기준)
    cp_tx_data: coverpoint t_item.tx_data_m {
        bins zero = {8'h00};
        bins max = {8'hff};
        bins others = {[8'h01 : 8'hfe]};
    }

    cx_mode_data: cross cp_cpol, cp_cpha, cp_tx_data;
endgroup
```

```
===== SPI Coverage Summary =====
 Overall Coverage : 100.0%
---------------------------------
 CPOL Coverage    : 100.0%
 CPHA Coverage    : 100.0%
 TX Data Coverage : 100.0%
---------------------------------
 Cross(Mode, Data): 100.0%
=================================
```

: UVM

SPI LoopBack - Coverage

Status	Bin Name	Type	At Least	Size	Hit Count
✓	mode0	User	1	1	1000
⊗	Ille...	others	User	1	1	0

Avg. Group Score:100.00% U+C:8 U:0 C:8 X:0
Avg. Group Inst. Score:100.00% U+C:8 U:0 C:8 X:0

Group	Score	Instances	U+C	U	C	X	Goal	Weight	AtLeast	PerInst	Overlap	AutoBinI	Missing	Comment
CG $unit::spi_coverage::cg_spi	100.00%		8	0	8	0	100%	1	1	0	1	64	64	
CP cp_cpha	100.00%		1	0	1	0	100%	1	1		1	0		
CP cp_cpol	100.00%		1	0	1	0	100%	1	1		1	0		
CP cp_tx_data	100.00%		3	0	3	0	100%	1	1		1	0		
CR cx_mode_data	100.00%		3	0	3	0	100%	1	1				0	

: UVM

I2C LoopBack

Trouble Shooting

: Trouble Shooting

- *문제 상황* SPI loopback을 UVM으로 검증하는 상황에서
  Master → Slave 에서 모든 경우 Fail 발생

- *원인 분석* Full Duplex 방식을 반영하기 위해
  Write와 Read를 동시에 수행하는 Scenario를 구성
  → tx data와 rx data를 비교하는 시점이 잘못됨

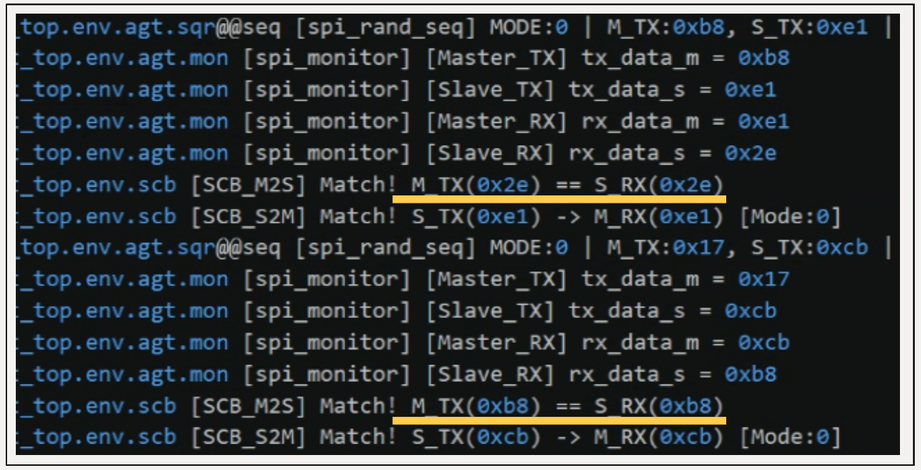
```
==========================================
       SPI SCOREBOARD SUMMARY
==========================================
 [Master->Slave]  Pass: 0, Fail: 100
 [Slave->Master]  Pass: 100, Fail: 0
------------------------------------------
 Total Successful Transfers: 100
==========================================
Test Result: FAILED (Check mismatches above)
```

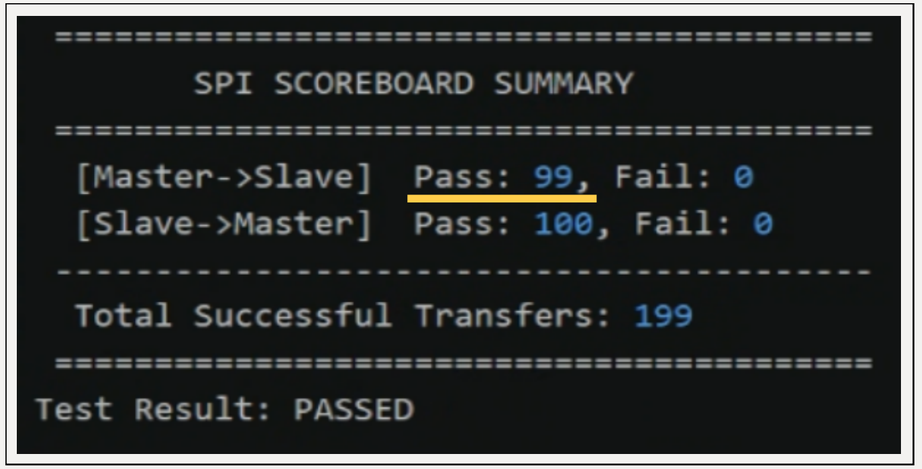
```
op.env.agt.sqr@@seq [spi_rand_seq] MODE:0 | M_TX:0xb8, S_TX:0xe1
top.env.agt.mon [spi_monitor] [Master_TX] tx_data_m = 0xb8
top.env.agt.mon [spi_monitor] [Slave_TX] tx_data_s = 0xe1
top.env.agt.mon [spi_monitor] [Master_RX] rx_data_m = 0xe1
top.env.agt.mon [spi_monitor] [Slave_RX] rx_data_s = 0x2e
_top.env.scb [SCB_M2S] Mismatch! M_TX(0xb8) != S_RX(0x2e)
top.env.scb [SCB_S2M] Match! S_TX(0xe1) == M_RX(0xe1)
op.env.agt.sqr@@seq [spi_rand_seq] MODE:0 | M_TX:0x17, S_TX:0xcb
top.env.agt.mon [spi_monitor] [Master_TX] tx_data_m = 0x17
top.env.agt.mon [spi_monitor] [Slave_TX] tx_data_s = 0xcb
top.env.agt.mon [spi_monitor] [Master_RX] rx_data_m = 0xcb
top.env.agt.mon [spi_monitor] [Slave_RX] rx_data_s = 0xb8
_top.env.scb [SCB_M2S] Mismatch! M_TX(0x17) != S_RX(0xb8)
top.env.scb [SCB_S2M] Match! S_TX(0xcb) == M_RX(0xcb)
```

: Trouble Shooting

- *문제 해결* master의 prev_tx_data 과 slave의 rx_data를 비교하는 방식으로 수정

- *느낀점* Full Duplex 방식에 집중한 나머지, slave에 먼저 write를 해놓는 부분을 간과
→ Scenario 구성 시, 꼼꼼하게 생각해야 함을 느낌





발표를 들어주셔서 감사합니다